DeepTrustxAI Kubernetes Security Foundations, Vulnerabilities and CloudGuard

DeepTrustxAI

Kubernetes Security Foundations, Vulnerabilities and CloudGuard

Separate two-day module-wise training programme

Kubernetes basics first · Cluster attack paths second · Live scanning and GRC third

---

Two-Day Delivery Model

Day 1: Understand Kubernetes

Containers, cluster architecture, objects, workloads, RBAC, namespaces, networking, storage, secrets, delivery and observability.

Day 2: Secure Kubernetes

How vulnerabilities enter, workload/node risk, RBAC/network/secret paths, supply chain, IaC drift, CloudGuard and capstone.

> Container → Cluster → Workload → Identity → Network/Data → Vulnerability → Evidence → Remediation

---

Prerequisites and Learning Outcomes

Recommended prerequisites

Linux CLI · TCP/IP · YAML · containers · basic IAM · basic cloud concepts

Participants will be able to

- Explain Kubernetes architecture and object relationships.
- Review workload, service-account, RBAC, network, storage and secret configuration.
- Recognize cluster vulnerability patterns and blast radius.
- Compare repository/IaC intent with live cluster state.
- Run CloudGuard's kubeconfig-based scanner in an authorized cluster.
- Map Kubernetes evidence to CISSP, CCSP and compliance controls.

CISSP and CCSP Domain Map

Kubernetes concept	CISSP alignment	CCSP alignment
Cluster architecture and isolation	D3 Architecture	D1 Concepts; D3 Infrastructure
Networking and service exposure	D4 Network Security	D3 Infrastructure
Service accounts and RBAC	D5 IAM	D3 Infrastructure; D5 Operations
Secrets, volumes and data	D2 Asset; D3 Architecture	D2 Cloud Data Security
Images, manifests and admission	D8 Software Development	D4 Cloud Application Security
Scanning, testing and drift	D6 Assessment	D3 Infrastructure; D5 Operations
Audit, incident and evidence	D7 Operations; D1 Risk	D5 Operations; D6 Compliance

Mapping uses the ISC2 CISSP outline effective 15 April 2024 and the CCSP outline available 20 June 2026. ISC2 has announced a revised CCSP outline effective 1 August 2026.

Kubernetes Lab Rules

- Use only the assigned disposable cluster, kubeconfig, context and namespace.
- Do not test production clusters or shared third-party environments.
- Do not expose real secrets or sensitive images.
- Apply only instructor-approved vulnerable manifests.
- Record every live change and restore the cluster after the exercise.
- Keep tenant, cluster, namespace and scan-job lineage in every finding.

ManifestApproved vulnerable YAML/IaC

ClusterDisposable kind, k3s or managed lab

ScannerKubeconfig-based inventory and checks

EvidenceLive object plus normalized finding

GRCControl, owner and remediation proof

Day 1 Agenda: Kubernetes Foundations

Module	Time	Outcome
K0 Orientation and cluster lab	30 min	Validate scope, context and evidence path
K1 Containers and Kubernetes basics	75 min	Explain images, containers and orchestration
K2 Cluster architecture and objects	75 min	Trace desired state through control plane and nodes
K3 Identity, RBAC and tenancy	75 min	Calculate and reduce effective access
K4 Networking, storage and secrets	60 min	Draw traffic/data boundaries
K5 Secure delivery and observability	75 min	Review supply-chain and audit controls

---

K0

Orientation and Cluster Lab

Scope, kubeconfig, namespaces, tenant and evidence flow

CISSP D1/D6 · CCSP D5/D6 · 30 minutes

K0 Exercise: Validate the Lab

Check	Expected result
Kubeconfig/context	Assigned disposable cluster only

Check	Expected result
Namespace	Team namespace and labels visible
Identity	Least-privilege training account/service account
Scanner	Health check and credential validation succeed
Evidence	Dry-run result appears under the correct tenant/cluster
Cleanup	Reset and TTL path confirmed

K1

Containers and Kubernetes Basics

Understand the workload unit before securing the cluster

CISSP D3/D8 · CCSP D1/D3/D4 · 75 minutes

From Source to Running Pod

SourceApplication, Dockerfile and dependency definition

ImageImmutable layers, metadata and software content

RegistryStorage, access, signing and scanning

PodOne or more containers plus identity, network and volumes

ControllerDesired state, scaling, rollout and recovery

Security boundary: the container is not a complete isolation boundary; node and cluster controls still matter.

Core Container Concepts

Image

Packaged filesystem, metadata and entry point. Risk includes vulnerable packages and embedded secrets.

Container

Runtime process with namespaces, cgroups, capabilities, mounts, identity and network.

Orchestrator

Schedules, reconciles, exposes, updates and recovers workloads across nodes.

Secure defaults

Non-root · minimal image · read-only filesystem · dropped capabilities · resource limits · explicit identity

---

K2

Cluster Architecture and Objects

Trace desired state through control plane, nodes and workloads

CISSP D3/D4 · CCSP D1/D3 · 75 minutes

Kubernetes Cluster Architecture

Control plane

API server, etcd, scheduler and controller manager accept desired state, store it, schedule work and reconcile changes.

Worker node

Kubelet, container runtime and networking components run pods and report status.

Component	Security question
API server	Who can authenticate and which requests are authorized/admitted?
etcd	Is cluster state encrypted, restricted and backed up?
Scheduler/controllers	Can configuration or identity influence placement and reconciliation?
Kubelet/runtime	Can a workload reach node-level resources or unsafe runtime interfaces?

Kubernetes Object Relationships

DeploymentDesired application rollout and replicas

ReplicaSetMaintains the requested pod count

PodScheduling and shared execution unit

ServiceStable virtual access to selected pods

Ingress/GatewayExternal or cross-service routing

Other controllers: StatefulSet · DaemonSet · Job · CronJob

K2 Exercise: Trace Desired State

Given a deployment and service manifest:

1. Identify the controller, selector and pod template.
2. Determine where the workload can be scheduled.
3. Identify its service exposure and DNS name.
4. Identify attached service account, volumes and configuration.
5. Describe what the controllers do when a pod fails.

K3

Identity, RBAC and Tenancy

Understand who can perform which action on which Kubernetes object

CISSP D5 · CCSP D3/D5 · 75 minutes

Kubernetes Identity and Authorization

PrincipalUser, group or service account

AuthenticationCertificate, token, OIDC or cloud identity

RoleAPI groups, resources, verbs and names

BindingSubject receives Role or ClusterRole

AdmissionPolicy accepts, mutates or rejects the request

Scope matters: Role/RoleBinding is namespaced; ClusterRole/ClusterRoleBinding may affect the cluster.

K3 Exercise: Calculate Effective Access

Review a prepared service account and answer:

- Which Roles and ClusterRoles are bound?
- Which verbs and resources are allowed?
- Can it create workloads with a different service account?
- Can it read Secrets or modify RBAC?
- Is the token automatically mounted?
- Can the permission be reduced to namespace/resource/name scope?

Output: effective-access graph and least-privilege binding diff.

K4

Networking, Storage and Secrets

Define east-west, north-south and data boundaries

CISSP D2/D3/D4 · CCSP D2/D3 · 60 minutes

Cluster Network and Data Concepts

Networking

Pod network, CNI, DNS, ClusterIP, NodePort, LoadBalancer, ingress/gateway and NetworkPolicy.

Storage

Volumes, persistent volumes, claims, storage classes, snapshots and provider-backed disks/files.

Configuration

ConfigMaps, Secrets, environment variables, projected volumes and external secret stores.

> A Service provides reachability. A NetworkPolicy defines permitted traffic only when the CNI enforces it.

K4 Exercise: Draw Allowed Traffic

For the lab application:

- Mark ingress, service and pod traffic.
- Identify namespaces and expected east-west communication.
- Define default-deny and explicit allow NetworkPolicies.
- Identify persistent and ephemeral data.
- Identify every Secret consumer and remove unnecessary mounts.

K5

Secure Delivery and Observability

Control images and manifests before deployment, then monitor runtime state

CISSP D6/D7/D8 · CCSP D3/D4/D5 · 75 minutes

Kubernetes Delivery Control Layers

Build

Minimal image, pinned dependencies, SBOM, secret scanning and reproducibility.

Registry

Access control, scanning, signing, retention and immutable tags/digests.

Admission

Validate signatures, workload policy, labels, limits, identity and approved registries.

Runtime

Audit logs, events, metrics, network/runtime monitoring, drift and incident evidence.

IaC/GitOps: repository intent + review + admission + live-state drift detection

---

Day 1 Checkpoint

Participants should now be able to explain:

- How container images become running workloads

- How the Kubernetes control plane stores and reconciles desired state
- How Pods, controllers, Services and ingress relate
- How service accounts, Roles and bindings create effective access
- How network, storage, Secrets, admission and observability controls work
- Which CISSP and CCSP domains relate to each concept

Day 2 question: how do weak manifests, exceptions and drift become exploitable cluster paths?

Day 2 Agenda: Vulnerabilities and Solution

Module	Time	Outcome
K6 Vulnerability lifecycle	75 min	Explain how weaknesses enter and persist
K7 Workload and node risk	75 min	Prioritize runtime escape/blast radius
K8 RBAC, network and secret paths	60 min	Connect identity and exposure findings
K9 Supply chain and IaC drift	75 min	Compare repository intent with live state
K10 CloudGuard scanner and GRC	45 min	Run live scan and ingest evidence
K11 Capstone	60 min	Scan, remediate and prove closure

K6

How Kubernetes Vulnerabilities Creep In

Follow weak YAML, exceptions and drift into runtime

CISSP D3/D6/D8 · CCSP D3/D4/D5 · 75 minutes

Kubernetes Vulnerability Introduction

Cause	Example	Preventive control	Detection/evidence
Copied manifest	Root/privileged/hostPath remains	Approved secure template	IaC finding and diff
Temporary exception	Admission bypass never expires	Time-bound exception	Admission/audit event
Broad identity	Cluster binding used for convenience	Least-privilege RBAC	Effective access finding
Troubleshooting exposure	NodePort/LoadBalancer remains	Change approval and expiry	Service inventory
Missing segmentation	New namespace lacks policy	Default-deny baseline	NetworkPolicy finding
Manual change	Live state differs from Git	GitOps/reconciliation	Drift evidence
Floating image	Unreviewed image version runs	Digest/signature policy	Image and deployment proof

---

Common Kubernetes Vulnerability Categories

Workload

Privileged mode, root, host namespaces, hostPath, capabilities and writable filesystem.

Identity

Broad service account, cluster binding, token auto-mount and RBAC escalation path.

Exposure

NodePort/LoadBalancer, permissive ingress, missing NetworkPolicy and unsafe DNS trust.

Data

Unnecessary Secret mounts, weak etcd/storage protection and broad namespace visibility.

Supply chain

Vulnerable image, mutable tag, untrusted registry, unsigned artifact and leaked token.

Operations

Missing audit logs, owner, limits, backup, drift monitoring and remediation deadline.

K7

Workload and Node-Risk Patterns

Prioritize controls by escape potential and node blast radius

CISSP D3/D7 · CCSP D3/D4/D5 · 75 minutes

Workload Risk Matrix

Setting	Risk	Safer direction
`privileged: true`	Broad device/kernel access	Disable; grant only required capabilities
`hostPath`	Node filesystem exposure	Avoid or tightly restrict path/read-only use
`hostPID/hostNetwork`	Host process/network visibility	Use isolated pod namespaces
Run as root	Higher impact after compromise	Enforce non-root UID/GID
Added capabilities	Expanded kernel operations	Drop all; add only required capability
Writable root filesystem	Persistence and tampering	Read-only root plus explicit writable volumes
No limits	Resource exhaustion/noisy neighbour	Requests, limits and quotas

---

K7 Exercise: Prioritize the Manifests

Review prepared manifests and rank findings by:

1. Can the workload reach host resources?
2. Can it influence other workloads or namespaces?
3. Which service account and secrets are available?
4. Is the service externally reachable?
5. Is the image trusted and reproducible?
6. What evidence proves the risk and the fix?

K8

RBAC, Network and Secret Attack Paths

Connect service exposure to identity and cluster impact

CISSP D2/D4/D5 · CCSP D2/D3/D5 · 60 minutes

Kubernetes Attack Path

Exposed serviceInternet or broad internal reachability

WorkloadApplication weakness or unsafe runtime

Service accountMounted token and effective RBAC

Cluster/data targetSecrets, workloads, RBAC or node resource

ImpactCross-namespace access, persistence or data disclosure

> Break the path with exposure control, workload hardening, least-privilege RBAC, secret minimization and audit evidence.

K9

Supply Chain and IaC Drift

Prove that the running cluster matches reviewed intent

CISSP D6/D8 · CCSP D3/D4/D5 · 75 minutes

Repository-to-Runtime Evidence

Source/IaCDockerfile, manifest, Helm/Kustomize or Terraform

PipelineBuild, scan, sign, attest and approve

AdmissionValidate image, identity, policy and labels

Live clusterActual object, image digest, RBAC and exposure

Drift evidenceExpected versus observed field and owner

Evidence: commit · build result · image digest · admission decision · live object · drift finding

---

K9 Exercise: Compare IaC and Live State

Compare repository manifests with the assigned cluster:

- Image tag/digest
- Security context and capabilities
- Service account and token auto-mount
- Service type and ports
- NetworkPolicy presence
- Secret/configuration references
- Labels, owner and environment

Create one normalized drift finding with tenant, cluster, namespace, object, field, expected value, observed value and remediation.

K10

How CloudGuard Helps Kubernetes

Connect through kubeconfig, scan live state and map evidence

CISSP D6/D7 · CCSP D3/D5/D6 · 45 minutes

CloudGuard Kubernetes Scanner Coverage

Area	Live checks	Evidence outcome
Inventory	Pods, services, namespaces, deployments, jobs and cronjobs	Tenant/cluster asset inventory
Workloads	Privilege, host settings, capabilities, root, mounts and secrets	Workload-risk findings
RBAC	RoleBindings and ClusterRoleBindings	Effective access and broad binding findings
Network	NodePort/LoadBalancer and NetworkPolicy coverage	Exposure and segmentation findings
IaC drift	Live resources compared with Kubernetes manifests	Expected-versus-observed evidence
Managed context	EKS/GKE/AKS metadata when cloud credentials exist	Provider/cluster enrichment

---

Kubernetes Evidence Flow

KubeconfigValidated credential reference and context

Scan jobProvider=kubernetes and tenant lineage

Live APIsCore, apps, batch, RBAC and networking objects

EvidenceRaw object reference plus normalized finding

GRCControl, owner, remediation, rescan and closure

The portal queues and displays work; scanner workers run separately.

K10 Exercise: Scan and Prove One Fix

1. Validate the assigned kubeconfig and context.
2. Start a Kubernetes scan job.
3. Review workload, service, RBAC and NetworkPolicy findings.
4. Compare one object with its IaC manifest.
5. Select the highest-risk connected path.
6. Apply an approved remediation.
7. Rescan and attach closure evidence to the control.

Kubernetes Hardening Model

Prevent

Secure templates, trusted images, admission policy, least privilege and default-deny network.

Detect

IaC scan, image scan, live-cluster scan, audit logs, runtime monitoring and drift.

Respond

Contain workload, rotate identity/secret, remove exposure and restore reviewed state.

Prove

Manifest diff, live object, scan job, evidence hash, owner, due date and rescan.

K11 Kubernetes Capstone

A disposable cluster contains intentional weaknesses in workloads, RBAC, Services, NetworkPolicies, Secrets, images and IaC drift.

Teams must:

1. Identify the highest-risk cluster attack path.
2. Validate it through live objects and effective access.
3. Run the CloudGuard Kubernetes scanner.
4. Reconcile scanner findings and IaC drift.
5. Map controls, owners and remediation.
6. Remediate selected findings and prove closure through rescan.

Official Mapping References

- ISC2 CISSP Exam Outline, effective 15 April 2024: isc2.org/certifications/cissp/cissp-certification-exam-outline
- ISC2 CCSP Exam Outline and announced 1 August 2026 revision: isc2.org/certifications/ccsp/ccsp-certification-exam-outline
- DeepTrustxAI official logo: deeptrustxai.com/images/darkmode.png

This workshop supports certification-domain understanding but is not an official ISC2 course or endorsement.

DeepTrustxAI

Understand the Cluster. Find the Path. Prove the Fix.

CloudGuard connects Kubernetes live state and IaC drift to tenant-safe evidence, controls and remediation.